Vérification Formelle du Protocole TCP avec UPPAAL

Modélisation, Analyse de Sécurité et Validation Formelle

Protocole étudié	TCP -- RFC 9293 (IETF, Août 2022)
Outil de vérification	UPPAAL 5.0 / UPPAAL-SMC
Type d'analyse	Vérification formelle par automates temporisés
Module	Sécurité des protocoles IoT et systèmes distribués
Encadrante	Mme Amina Jandoubi
Année universitaire	2025 -- 2026
Membre	Yemen Taoufiq Mahmoud, Yomna Hlaili, Youssef Jbeniani

Table des matières

1.	Contexte et objectifs du projet	3
2.	Spécification officielle du protocole TCP	3
3.	Répartition des tâches	4
4.	Extraction des comportements pertinents	4
5.	Modélisation UPPAAL	6
6.	Vérifications générales des propriétés	9
7.	Attaques CVE et modélisation formelle	0
8.	Requêtes UPPAAL et résultats de vérification	3
9.	Contre-mesures et validation formelle	4
10	Conclusion et perspectives	1

1. Contexte et objectifs du projet

Les protocoles de communication utilisés dans l'Internet des objets (IoT) et les systèmes distribués sont régulièrement exposés à des vulnérabilités documentées dans la base CVE (Common Vulnerabilities and Exposures). Ce projet a pour objectif d'appliquer les techniques de vérification formelle à un protocole réel afin d'en analyser rigoureusement les propriétés de sécurité et de comportement.

Le protocole retenu pour cette étude est le **Transmission Control Protocol (TCP)**, protocole de transport fondamental de l'Internet, dont la spécification de référence est la RFC 9293 publiée par l'IETF en août 2022. TCP a été choisi en raison de sa criticité dans les communications réseau, de la richesse de sa machine à états, et de l'existence de plusieurs vulnérabilités historiques et contemporaines bien documentées.

Objectifs principaux : (1) Construire un modèle formel de TCP sous forme de réseau d'automates temporisés dans UPPAAL. (2) Vérifier les propriétés générales du protocole (absence de deadlock, reachability, safety, liveness). (3) Modéliser deux attaques issues de la base CVE et les vérifier formellement. (4) Proposer des contre-mesures et valider leur efficacité dans le modèle.

2. Spécification officielle du protocole TCP

La spécification officielle retenue pour ce projet est la **RFC 9293**, publiée par l'Internet Engineering Task Force (IETF) en août 2022. Ce document constitue la mise à jour consolidée de la spécification originale RFC 793 (1981), en y intégrant l'ensemble des corrections, clarifications et améliorations apportées depuis lors, notamment les RFC 879, 2873, 6093, 6429, 6528, 6691 et 5961.

Document	Référence complète
Référence principale	RFC 9293 -- Transmission Control Protocol (TCP), IETF, Août 2022
Référence historique	RFC 793 -- Original TCP Specification, IETF, Septembre 1981
Timers TCP	RFC 6298 -- Computing TCP's Retransmission Timer, IETF, Juin 2011
Attaque SYN Flood	RFC 4987 -- TCP SYN Flooding Attacks and Common Mitigations, IETF, 2007
Contremesure RST	RFC 5961 -- Improving TCP's Robustness to Blind In-Window Attacks, 2010

TCP est un protocole de transport orienté connexion qui garantit la livraison fiable, ordonnée et sans duplication des données. Il opère en mode full-duplex et repose sur un mécanisme de numérotation de séquence, d'acquittements cumulatifs, de retransmission sur temporisation et de contrôle de flux par fenêtre glissante.

3. Répartition des tâches

Membre	Responsabilités
Yemen	Lecture RFC 9293, extraction des comportements, machine à états TCP
Yemen	Modélisation UPPAAL : template Client et Server, déclarations globales
Yomna	Vérification des propriétés générales, rédaction des requêtes CTL
Youssef	Modélisation des attaques CVE, automates d'attaque SYN Flood et RST
Yomna	Contre-mesures, validation formelle, rédaction du rapport

Note : Bien que les tâches soient réparties, chaque membre a participé à la relecture croisée du modèle UPPAAL et à la validation des résultats de vérification. La présentation orale est préparée collectivement.

4. Extraction des comportements pertinents

4.1 Types de messages (segments TCP)

La RFC 9293 définit les types de segments suivants, chacun identifié par des bits de contrôle dans l'en-tête TCP :

Segment	Rôle dans le protocole
SYN	Demande d'établissement de connexion -- synchronisation des numéros de séquence
SYN-ACK	Réponse du serveur au SYN -- acceptation et synchronisation en retour
ACK	Acquittement de données ou de segments de contrôle reçus
FIN	Demande de fermeture de la connexion dans un sens (half-close)
FIN-ACK	Acquittement du FIN reçu de l'autre partie
RST	Réinitialisation immédiate de la connexion -- fermeture brutale
DATA	Segment portant des données applicatives (avec ACK implicite)

4.2 États et transitions -- Machine à états TCP

TCP est un protocole à états. Chaque extrémité maintient un état courant qui évolue selon les segments reçus et les actions de l'application. La RFC 9293 définit les états suivants :

État	Description
CLOSED	État initial. Aucune connexion active ou en cours.
LISTEN	Serveur en attente passive de demande de connexion entrante.
SYN_SENT	Client ayant envoyé un SYN, en attente du SYN-ACK du serveur.
SYN_RECEIVED	Serveur ayant reçu un SYN et envoyé un SYN-ACK, en attente du ACK final.
SYN_ACK_SENT	Serveur ayant envoyé un SYN-ACK en attente d'une confirmation de la part du client
ESTABLISHED	Connexion établie. Les données peuvent circuler dans les deux sens.
FIN_WAIT_1	Client ayant initié la fermeture, en attente d'un ACK de son FIN.
FIN_WAIT_2	Client ayant reçu l'ACK de son FIN, en attente du FIN du serveur.
TIME_WAIT	Client attendant 2xMSL avant fermeture définitive (garantie de livraison).
CLOSE_WAIT	Serveur ayant reçu le FIN du client, en attente de fermeture applicative.
LAST_ACK	Serveur ayant envoyé son FIN, en attente du dernier ACK du client.

4.3 Contraintes temporelles -- Timers TCP

Timer	Rôle	Valeur réelle	Abstraction UPPAAL
Timeout SYN	Délai avant retransmission du SYN si aucun SYN-ACK n'est reçu.	~1s (double à chaque tentative)	SYN_TIMEOUT = 5 (abstrait)
Timer TIME_WAIT	Durée minimale d'attente avant fermeture définitive.	2 x MSL (60–120s)	TIME_WAIT_DUR = 10 (abstrait)
Timer RTO	Délai de retransmission des données applicatives.	Dynamique (min 200ms)	Non modélisé (hors scope)

Justification des abstractions temporelles : Les valeurs réelles des timers TCP (MSL = 60s, timeout SYN = 1s) sont remplacées par de petites valeurs entières dans UPPAAL. Cette abstraction est valide car la vérification formelle s'intéresse aux propriétés *comportementales* du protocole (ordre des transitions, respect des états), et non aux performances temporelles absolues.

5. Modélisation UPPAAL

5.1 Déclarations globales

Les déclarations globales définissent l'ensemble des variables, canaux et constantes partagés entre tous les templates du modèle :

```
// ■■ CANAUX DE SYNCHRONISATION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
chan syn;         // Client envoie SYN, Serveur reçoit
chan syn_ack;     // Serveur envoie SYN-ACK, Client reçoit
chan ack;         // Client envoie ACK final, Serveur reçoit
chan fin;         // Client envoie FIN, Serveur reçoit
chan ack_fin;     // Serveur acquitte le FIN du client
chan fin2;        // Serveur envoie son propre FIN
chan ack_fin2;    // Client envoie le dernier ACK
chan rst;         // Segment RST (utilisé par l'attaquant)

bool syn_received_flag = false;
clock c;

// ■■ VARIABLES GLOBALES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
int  syn_count = 0;          // Compteur de retransmissions SYN
int  connection_alive = 0;   // 1 = connexion ESTABLISHED active
bool rst_injected = false;   // Indicateur d'attaque RST

// ■■ CONSTANTES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const int MAX_SYN       = 3;
const int SYN_TIMEOUT   = 5;    // Tentatives SYN maximales
                                // Timeout SYN (abstrait)
const int TIME_WAIT_DUR = 10;   // Durée TIME_WAIT (abstrait)
```

5.2 Template Client

Le template Client modélise le côté initiateur de la connexion TCP (active opener). Il dispose d'une horloge locale **c** pour gérer les timeouts de retransmission et de TIME_WAIT.

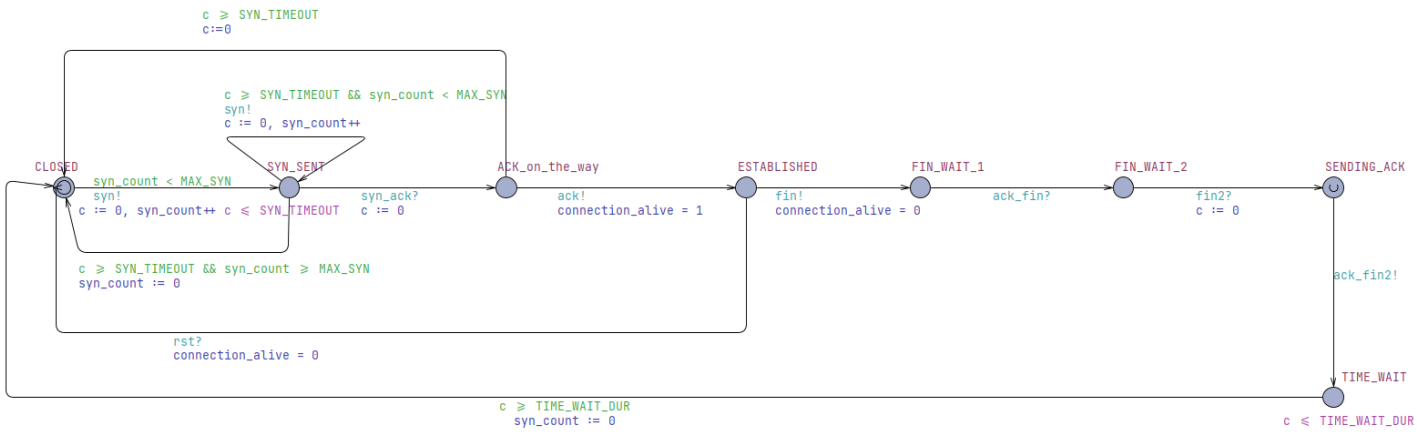


Figure 1 -- Automate Client

Figure 1 : Automate temporisé du Client TCP modélisant l'établissement, le transfert de données et la fermeture de connexion.

Transition	Garde	Sync	Mise à jour
CLOSED -> SYN_SENT	syn_count < MAX_SYN	syn!	c := 0, syn_count++
SYN_SENT → ACK_on_the_way	(aucune)	syn_ack?	c:=0,
ACK_on_the_way → ESTABLISHED	(aucune)	*ack!*	connection_alive = 1
SYN_SENT -> SYN_SENT	c>=SYN_TIMEOUT and syn_count<MAX_SYN	syn!	c:=0, syn_count++
SYN_SENT -> CLOSED	c>=SYN_TIMEOUT and syn_count>=MAX_SYN	--	syn_count := 0
ESTABLISHED -> FIN_WAIT_1	(aucune)	fin!	connection_alive = 0
ESTABLISHED -> CLOSED	(aucune)	rst?	connection_alive = 0
FIN_WAIT_1 -> FIN_WAIT_2	(aucune)	ack_fin?	--
FIN_WAIT_2 → SENDING_ACK	*(aucune)*	*fin2?*	*c := 0*
SENDING_ACK -> TIME_WAIT	(aucune)	ack_fin2!?	c := 0
TIME_WAIT -> CLOSED	c >= TIME_WAIT_DUR	--	syn_count := 0

5.3 Template Server

Le template Server modélise le côté passif (passive opener). Il démarre dans l'état LISTEN et gère la file des demi-connexions (half-open connections) via un compteur **queue**, essentiel pour la modélisation de l'attaque SYN Flood.

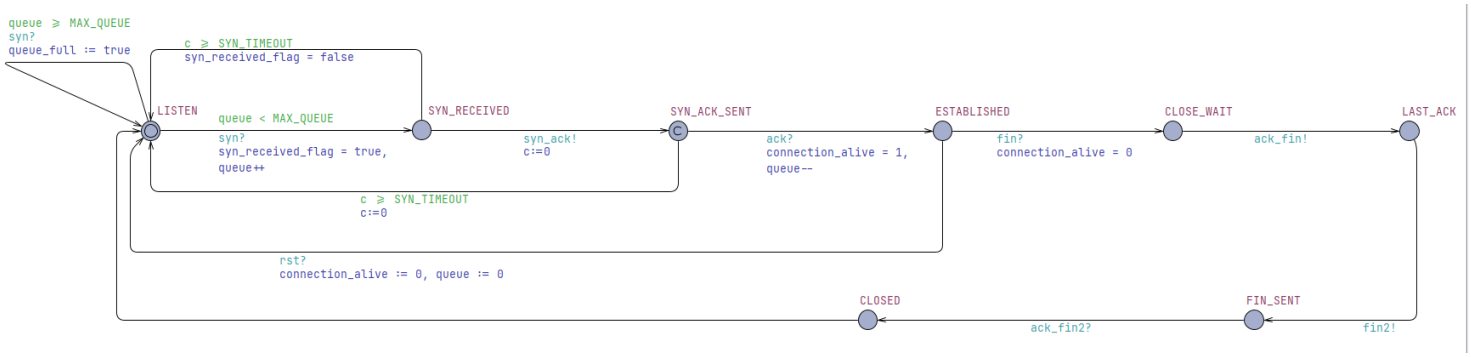


Figure 2 -- Automate Server

Figure 2 : Automate temporisé du Serveur TCP avec gestion de la file de connexions.

Transition	Garde	Synchro	Mise à jour
LISTEN -> SYN_RECEIVED	queue < MAX_QUEUE	syn?	s := 0, queue++
SYN_RECEIVED -> SYN_ACK_SENT	(aucune)	syn_ack!	c:=0
SYN_ACK_SENT -> ESTABLISHED	(aucune)	ack?	connection_alive = 1, queue--
SYN_ACK_SENT -> LISTEN	s >= SYN_TIMEOUT	--	connection_alive := 0, queue := 0
ESTABLISHED -> CLOSE_WAIT	(aucune)	fin?	connection_alive = 0
ESTABLISHED -> LISTEN (RST)	(aucune)	rst?	connection_alive := 0, queue := 0
CLOSE_WAIT -> LAST_ACK	(aucune)	ack_fin!	--
LAST_ACK -> FIN_SENT	(aucune)	fin2!	--
FIN_SENT -> CLOSED	(aucune)	ack_fin2?	--
LISTEN -> LISTEN (rejet)	queue>=MAX_QUEUE	syn?	queue_full := true

5.4 Déclaration système

```
// Instanciation des templates
Client_inst = Client();
Server_inst = Server();

// Déclaration du système (sans attaquants -- modèle nominal)
system Client_inst, Server_inst;
```

6. Vérifications générales des propriétés

Avant d'introduire les attaquants, nous vérifions que le modèle nominal de TCP satisfait les propriétés fondamentales. Ces vérifications valident la correction du modèle et constituent la base de référence pour l'analyse de sécurité.

6.1 Opérateurs CTL utilisés

Opérateur	Signification	Exemple
A[]	Pour tous les chemins, toujours (invariant global)	A[] not deadlock
E<>	Il existe un chemin où éventuellement (atteignabilité)	E<> Client_inst.ESTABLISHED
A<>	Pour tous les chemins, éventuellement (vivacité)	A<> Client_inst.CLOSED
-->	Conduit à (p --> q : si p alors éventuellement q)	Client_inst.SYN_SENT --> Client_inst.ESTABLISHED

6.2 Requêtes et résultats -- modèle nominal

```
// Q1 -- Absence de deadlock (première vérification obligatoire) A[] not deadlock

// Q2 -- Connexion établie atteignable des deux côtés
E<> Client_inst.ESTABLISHED and Server_inst.ESTABLISHED

// Q3 -- Fermeture propre atteignable
E<> Client_inst.CLOSED and Server_inst.LISTEN

// Q4 -- L'état TIME_WAIT est atteignable
E<> Client_inst.TIME_WAIT
```

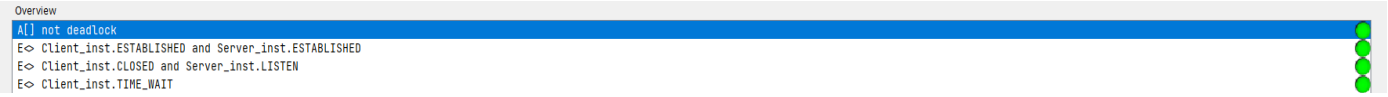


Figure 3 : Capture de l'onglet Verifier d'UPPAAL montrant les résultats des requêtes Q1 à Q6 sur le modèle nominal (sans attaquant).

Interprétation : Les requêtes Q1 à Q6 doivent toutes retourner **« Property is satisfied »** dans UPPAAL. En particulier, Q1 (absence de deadlock) et Q5 (vivacité) confirment que le modèle représente fidèlement le comportement attendu de TCP selon la RFC 9293.

7. Attaques CVE et modélisation formelle

Distinction fondamentale : Le projet exige deux types d'attaques. (1) **Attaque liée à la spécification** : la vulnérabilité est inhérente à la conception du protocole -- même une implémentation parfaite de la RFC resterait vulnérable. (2) **Attaque liée à l'implémentation** : la RFC est correcte, mais des choix d'implémentation spécifiques créent la vulnérabilité.

7.1 Attaque 1 -- SYN Flood (vulnérabilité de spécification)

Référence CVE : CVE-1999-0116 -- documentée formellement dans la RFC 4987 (2007).

Principe de l'attaque : Le mécanisme du handshake à trois voies impose au serveur d'allouer des ressources mémoire (un Transmission Control Block -- TCB) dès la réception d'un SYN, avant que la connexion soit complète. Un attaquant envoie un flux massif de segments SYN avec des adresses IP source falsifiées (spoofées). Le serveur alloue un slot pour chaque SYN, attend un ACK qui ne viendra jamais, et sa file de demi-connexions (backlog) se remplit jusqu'à saturation. Les connexions légitimes sont alors rejetées.

Lien avec la RFC 9293 : La RFC 9293 (Section 3.10.7) impose que le serveur entre en état SYN_RECEIVED à la réception d'un SYN et alloue les ressources nécessaires. Cette exigence n'a pas été modifiée depuis la RFC 793 originale. La vulnérabilité est donc structurelle et inhérente à la spécification du handshake TCP.

Automate d'attaque : Attacker_SY

```
// Déclarations locales du template Attacker_SYN
int flood_count = 0;
const int FLOOD_LIMIT = 5;  // Représente la saturation de la file

// Transitions
// IDLE -> FLOODING     : syn!  / flood_count := 1
// FLOODING -> FLOODING : [flood_count < FLOOD_LIMIT] syn! / flood_count++
// FLOODING -> SUCCESS  : [flood_count >= FLOOD_LIMIT]
```

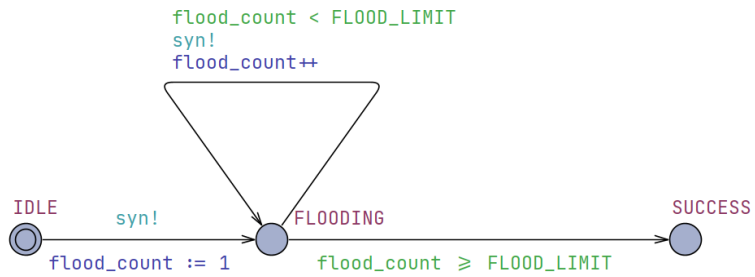


Figure 4 : Automate de l'attaquant SYN Flood modélisant l'envoi répété de segments SYN sans compléter le handshake.

7.2 Attaque 2 -- Injection RST (vulnérabilité d'implémentation)

Référence CVE : CVE-2004-0230 -- TCP RST Injection via large window size.**Principe de l'attaque :** Le segment RST permet la réinitialisation immédiate d'une connexion TCP établie. Les premières implémentations TCP acceptaient un RST si son numéro de séquence se trouvait *n'importe où* dans la fenêtre de réception (pouvant atteindre 65 535 octets). Un attaquant hors-chemin (off-path), connaissant les adresses IP et ports source/destination, peut deviner un numéro de séquence valide et injecter un RST pour terminer brutalement la connexion.

Impact réel : CVE-2004-0230 a affecté les sessions BGP (Border Gateway Protocol) s'exécutant sur TCP. Des sessions de routage inter-ISP longue durée pouvaient être interrompues, provoquant des perturbations sur des tables de routage internet.

Lien avec RFC 9293 : La RFC 9293 intègre la RFC 5961 qui introduit le mécanisme de Challenge ACK comme contre-mesure. Notre modèle démontre la vulnérabilité sur une implémentation RFC 793 de base, puis valide formellement la correction apportée par RFC 5961 / RFC 9293.

Automate d'attaque : Attacker_RST

```
// Déclarations locales du template Attacker_RST
clock a;  // Horloge de l'attaquant

// États : WAIT, OBSERVED, INJECTING, SUCCESS
// WAIT -> OBSERVED     : [connection_alive == 1] / a := 0
// OBSERVED -> INJECT   : [a >= 1]
// INJECT -> SUCCESS    : [connection_alive == 1] rst! / rst_injected := true
```

Figure 5 : Automate de l'attaquant RST modélisant l'observation puis l'injection d'un segment RST dans une connexion établie.

Déclaration système avec attaquants

```
Client_inst         = Client();
Server_inst         = Server();
Attacker_SYN_inst   = Attacker_SYN();
Attacker_RST_inst   = Attacker_RST();

system Client_inst, Server_inst, Attacker_SYN_inst, Attacker_RST_inst;
```

8. Requêtes UPPAAL et résultats de vérification

8.1 Requêtes d'attaque -- SYN Flood

```
// Q7 -- L'attaquant peut-il atteindre SUCCESS ?
E<> Attacker_SYN_inst.SUCCESS

// Q8 -- La file du serveur peut-elle se saturer ?
E<> Server_inst.queue_full

// Q9 -- Scénario combiné : attaque réussie ET client légitime bloqué
E<> Attacker_SYN_inst.SUCCESS and Client_inst.SYN_SENT and Server_inst.queue_full

// Q10 -- Safety violée : client bloqué alors que serveur est en LISTEN
E<> Server_inst.queue_full and Client_inst.CLOSED
```

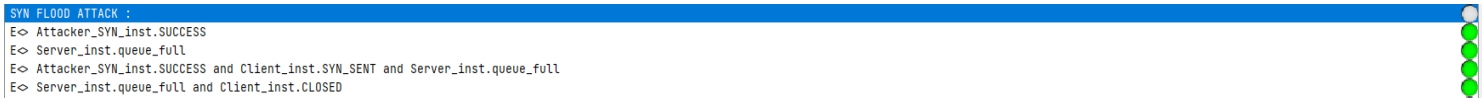


Figure 6 : Les quatre requêtes de l'attaque SYN Flood retournent « Property is satisfied », prouvant formellement la faisabilité de l'attaque.

8.2 Requêtes d'attaque -- Injection RST

```
// Q11 -- L'attaquant RST peut-il atteindre SUCCESS ?
E<> Attacker_RST_inst.SUCCESS

// Q12 -- Une connexion établie peut-elle être tuée par RST ?
E<> rst_injected and Client_inst.CLOSED and Server_inst.LISTEN

// Q13 -- Propriété critique : fermeture sans passage par FIN_WAIT_1
//        (preuve que l'attaque contourne la fermeture normale)
E<> rst_injected and not Client_inst.FIN_WAIT_1
```



Figure 7 : Les requêtes de l'attaque RST retournent « Property is satisfied », démontrant qu'une connexion TCP établie peut être terminée brutalement sans suivre la procédure de fermeture normale.

8.3 Tableau récapitulatif des résultats

N°	Requête (résumée)	Catégorie	Résultat	Interprétation
Q 1	A[] not deadlock	Nominale	OK Satisfaite	Modèle sans deadlock
Q 2	E<> Client.ESTABLISHED and Server.ESTABLISHED	Nominale	OK Satisfaite	Connexion atteignable
Q 3	E<> Client.CLOSED and Server.LISTEN	Nominale	OK Satisfaite	Fermeture propre possible
Q 7	E<> Attacker_SYN.SUCCESS	SYN Flood	OK Satisfaite	Attaque faisable
Q 9	E<> SUCCESS and Client.SYN_SENT and queue_full	SYN Flood	OK Satisfaite	Client bloqué par attaque
Q 11	E<> Attacker_RST.SUCCESS	RST Injection	OK Satisfaite	Attaque RST faisable
Q 13	E<> rst_injected and not Client.FIN_WAIT_1	RST Injection	OKSatisfaite	Fermeture normale bypassée

9. Contre-mesures et validation formelle

9.1 Contre-mesure 1 -- SYN Cookies (contre SYN Flood)

Principe : Les SYN Cookies, introduits par D.J. Bernstein (1996) et décrits dans la RFC 4987, permettent au serveur de répondre au SYN *sans allouer de mémoire*. Au lieu de créer un TCB immédiatement, le serveur encode toutes les informations de connexion nécessaires dans le numéro de séquence initial (ISN) de son SYN-ACK. Les ressources ne sont allouées qu'à la réception du ACK final, dont la validité est vérifiée par la valeur cryptographique encodée. Le handshake devient ainsi *stateless* côté serveur jusqu'à la confirmation.

Modélisation dans UPPAAL : Un booléen global **syn_cookie_enabled** active la contre-mesure. Lorsqu'il est vrai, la transition LISTEN -> SYN_RECEIVED du serveur ne modifie plus le compteur de file, rendant la saturation impossible.

```
// Ajout dans les déclarations globales
bool syn_cookie_enabled = true;

// Modification de la transition LISTEN -> SYN_RECEIVED du Serveur
// Garde modifiée :
// !syn_cookie_enabled || queue < MAX_QUEUE
// Mise à jour modifiée :
// s := 0, queue := (syn_cookie_enabled ? 0 : queue + 1)
```

Requêtes de validation :

```
// Après activation : syn_cookie_enabled = true

// QC1 -- L'attaque ne doit plus réussir (NOT satisfied attendu)
E<> Attacker_SYN_inst.SUCCESS and Server_inst.queue_full

// QC2 -- Le fonctionnement normal doit être préservé (satisfied attendu)
E<> Client_inst.ESTABLISHED and Server_inst.ESTABLISHED
```

F igur e 8 : Av ec les SY N Cookies activés, la requête d'attaque retourne « Property is NOT satisfied » (contre-mesure efficace), tandis que la connexion normale reste atteignable.

9.2 Contre-mesure 2 -- Challenge ACK / RFC 5961 (contre RST Injection)

Principe : La RFC 5961 (intégrée dans la RFC 9293) modifie le traitement des segments RST reçus hors séquence. Au lieu d'accepter tout RST dont le numéro de séquence est dans la fenêtre de réception, le serveur répond par un **Challenge ACK** si le numéro n'est pas exactement le prochain numéro attendu. Seul un RST avec le numéro exact est accepté immédiatement, réduisant la fenêtre d'attaque de 65 535 valeurs possibles à 1.

```
// Ajout dans les déclarations globales
bool rfc5961_enabled = true;
chan challenge_ack;

// Modification dans le template Client :
// ESTABLISHED -> CHALLENGING : [rfc5961_enabled] rst? / challenge_sent := true
// CHALLENGING -> ESTABLISHED : [rst_injected]   (RST attaquant rejeté)
// CHALLENGING -> CLOSED      : [!rst_injected] (RST légitime accepté)
```

Requêtes de validation :

```
// QC3 -- Avec RFC 5961 : l'attaque RST ne réussit plus (NOT satisfied attendu)
E<> Attacker_RST_inst.SUCCESS and rst_injected

// QC4 -- La connexion survit à la tentative d'attaque
A[] (rst_injected and rfc5961_enabled) imply Client_inst.ESTABLISHED
```

9 : La contre-mesure Challenge ACK (RFC 5961) prouve formellement que l'injection RST par un attaquant hors-chemin ne peut plus terminer une connexion TCP établie.

9.3 Tableau récapitulatif des contre-mesures

Attaque	Contre-mesure	Mécanisme de protection	Résultats UPPAAL
SYN Flood (CVE-1999-0116)	SYN Cookies (RFC 4987)	File stateless -- aucune allocation avant ACK final	QC1 : NOT satisfied QC2 : satisfied
RST Injection (CVE-2004-0230)	Challenge ACK (RFC 5961 / RFC 9293)	Numéro de séquence exact requis pour accepter RST	QC3 : NOT satisfied QC4 : satisfied

10. Conclusion et perspectives

Ce projet a permis d'appliquer la vérification formelle au protocole TCP, l'un des protocoles les plus fondamentaux de l'Internet, en utilisant l'outil UPPAAL. À partir de la spécification officielle RFC 9293, nous avons construit un modèle formel structuré, vérifié ses propriétés comportementales, modélisé deux attaques réelles issues de la base CVE, et validé formellement l'efficacité des contre-mesures correspondantes.

Résultats obtenus :

- Le modèle nominal de TCP satisfait toutes les propriétés générales : absence de deadlock, atteignabilité de l'état ESTABLISHED, fermeture propre, et vivacité.
- L'attaque SYN Flood (CVE-1999-0116) a été formellement démontrée comme faisable, avec preuve du blocage d'un client légitime lors de la saturation de la file serveur.
- L'attaque RST Injection (CVE-2004-0230) a été formellement démontrée comme permettant la terminaison brutale d'une connexion établie sans suivre le protocole de fermeture TCP.
- Les deux contre-mesures (SYN Cookies et Challenge ACK / RFC 5961) ont été modélisées et validées : les requêtes d'attaque retournent « Property is NOT satisfied » après activation, prouvant formellement leur efficacité.
- La vérification exhaustive par UPPAAL garantit que les résultats sont valides pour tous les chemins d'exécution possibles du modèle, et non pour un sous-ensemble.

Limites et perspectives :

Le modèle développé constitue une abstraction du protocole TCP. Certains aspects n'ont pas été modélisés dans le cadre de ce projet : le contrôle de congestion (RFC 5681), le mécanisme SACK (Selective Acknowledgment -- CVE-2019-11477), et les aspects de performance (débit, latence). Ces éléments constituent des perspectives d'extension naturelles pour des travaux futurs.

Par ailleurs, UPPAAL-SMC permettrait de quantifier la probabilité de succès des attaques sous des conditions probabilistes (pertes de paquets, comportement aléatoire de l'attaquant), offrant une dimension statistique complémentaire à la vérification exhaustive réalisée dans ce projet.